



Open-Ended Lab

FINITE STATE MACHINE

(OPEN-ENDED LAB)

RA: MS. MAHAM TABASSUM

Finite State Machine

- A **Finite State Machine (FSM)** is a model used to design and implement sequential logic circuits.
- FSM is a system that transitions between a finite number of states based on inputs and current states, making it fundamental for creating predictable, controlled processes in digital systems.
- FSM consists of two main components: a state register that holds the current state and a next-state logic that determines the next state based on inputs.

Finite State Machine

- There are two types of FSM:
 1. **Moore Machine**: In a Moore machine, outputs depend only on the current state, making its behavior easier to predict and design for certain applications.
 2. **Mealy Machine**: In a Mealy machine, outputs depend on both the current state and inputs, often resulting in more efficient designs but with slightly more complexity.
- **Examples**:
 - Moore Machine – Traffic Light System
 - Mealy Machine – Door lock system with the password

Example: Moore Machine

- Scenario: Traffic Light Controller
- Behavior: The output (light) depends only on the current state.

Current State	Output (Light)	Next State
Green	Green	Yellow
Yellow	Yellow	Red
Red	Red	Green

- Conclusion: The light (output) changes based on the current state & timer (not an input).
- Output is associated with the current state only.

Example: Mealy Machine

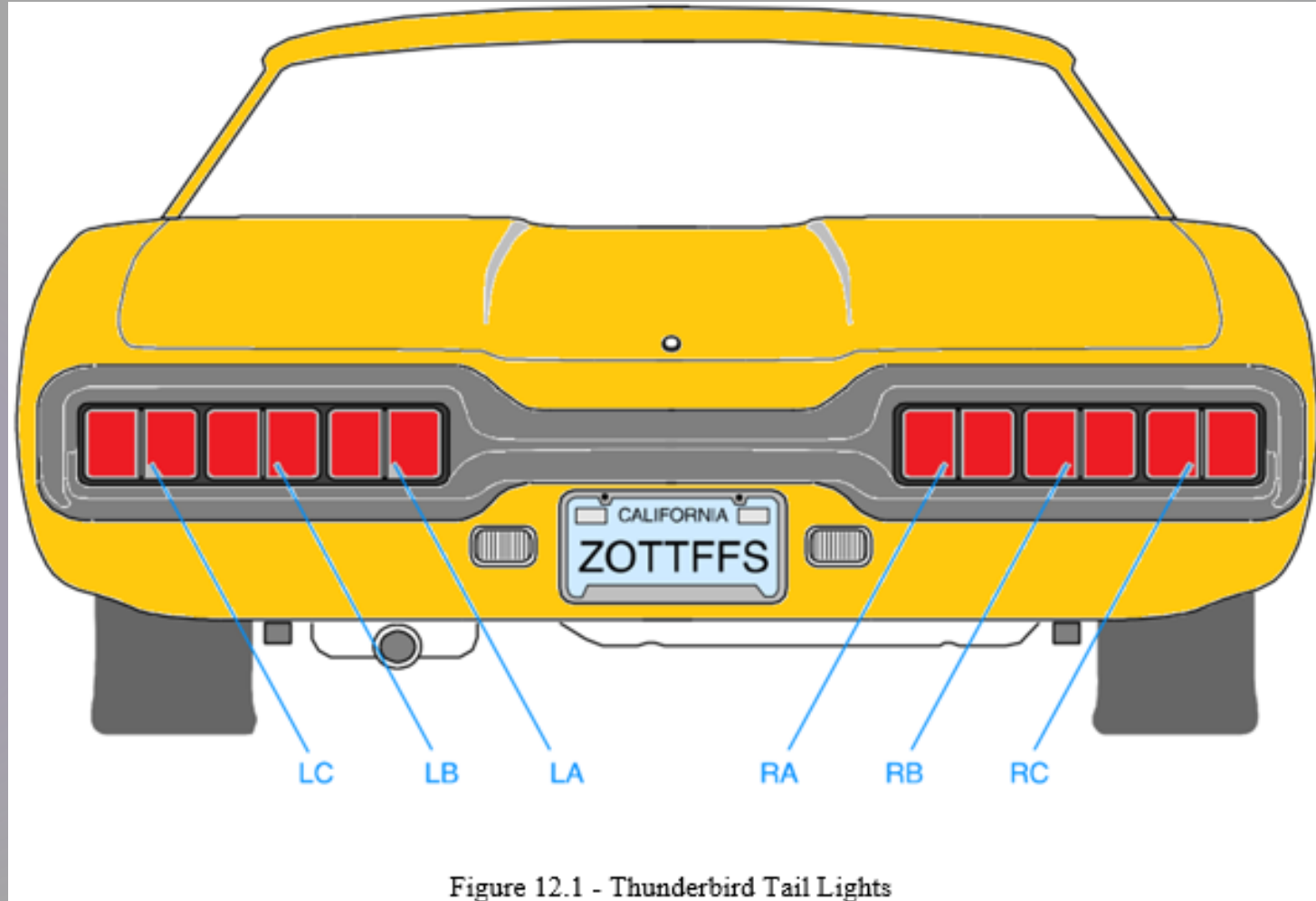
- Scenario: Door Lock System with keycard
- Behavior: The door unlocks (output) only if a card is swiped *and* it's valid (input).

Current State	Input (Card)	Output (Unlock?)	Next State
Locked	Valid Card	Unlock (Yes)	Unlocked
Locked	Invalid Card	Not unlock (No)	Locked
Unlocked	-	-	Locked (after delay)

- Conclusion: The output (unlock signal) depends on both the current state *and* the input (valid/invalid card).
- Output is associated with transitions (state + input).

Lab Objective

- Design a FSM to control the tail lights of a 1965 Ford Thunderbird.
- There are three lights on each side that operate in sequence to indicate the direction of a turn. That is **left turns** and **right turns**.



Lab Objective

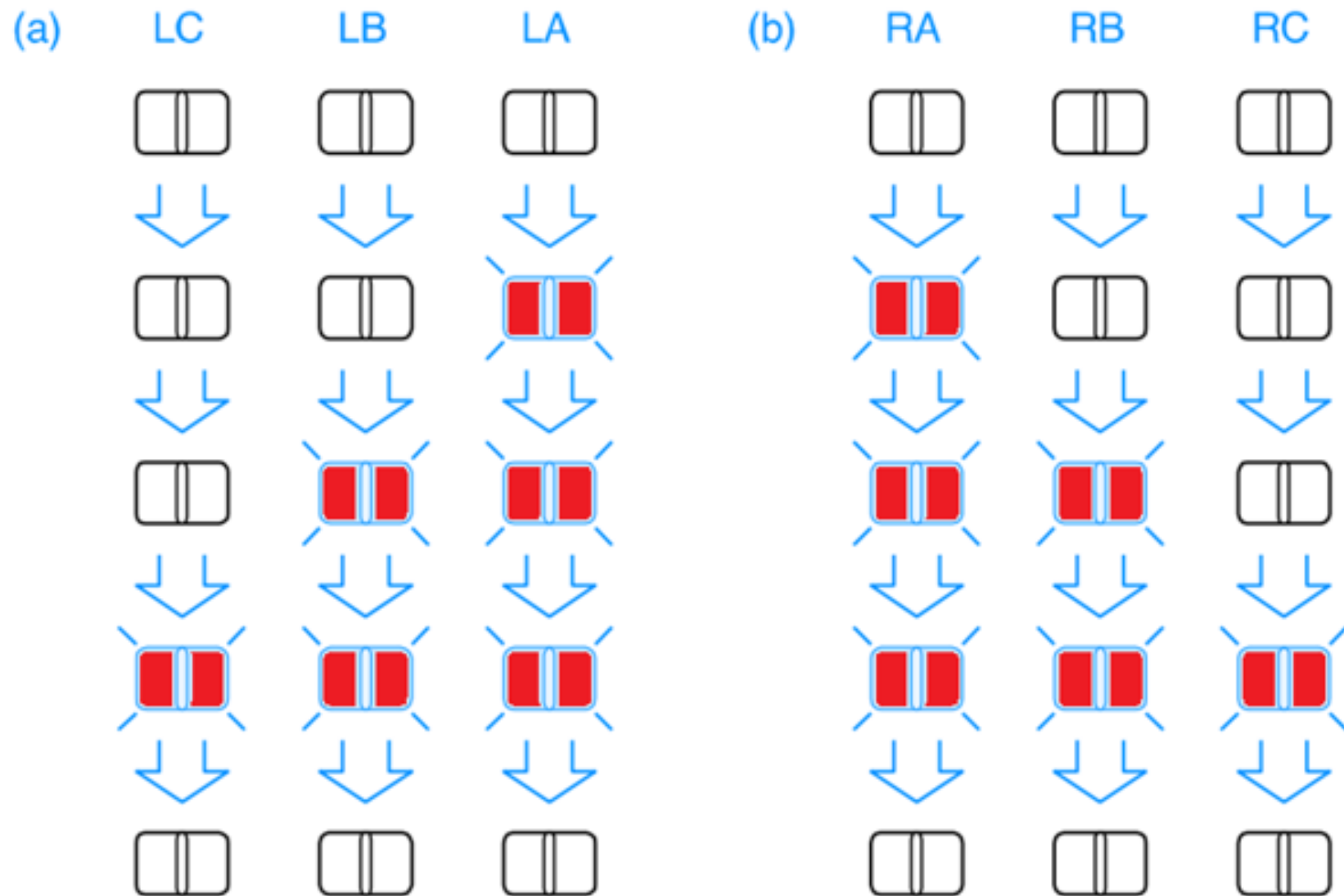


Figure 12.2 - Flashing Sequence

Design Requirements

- 3 Inputs of FSM → Left , Right , Reset
- 6 outputs of FSM → LA , LB, LC, RA, RB, RC
- On Reset → IDLE State → All lights off
- On pressing Left → you should see LA, then LA and LB, then LA, LB, and LC then all lights off.
- On pressing Right → you should see RA, then RA and RB, then RA, RB, and RC then all lights off
- Pattern should occur even if the button is released during sequence.
- If button is still pressed when the light turn off, then pattern should repeat.
- If user press both (L&R) simultaneously → make a choice by yourself for states.
- Decide what flipflops to choose for implementation of FSM.
- Decide what binary codes to choose for assigning binary value to state variables.

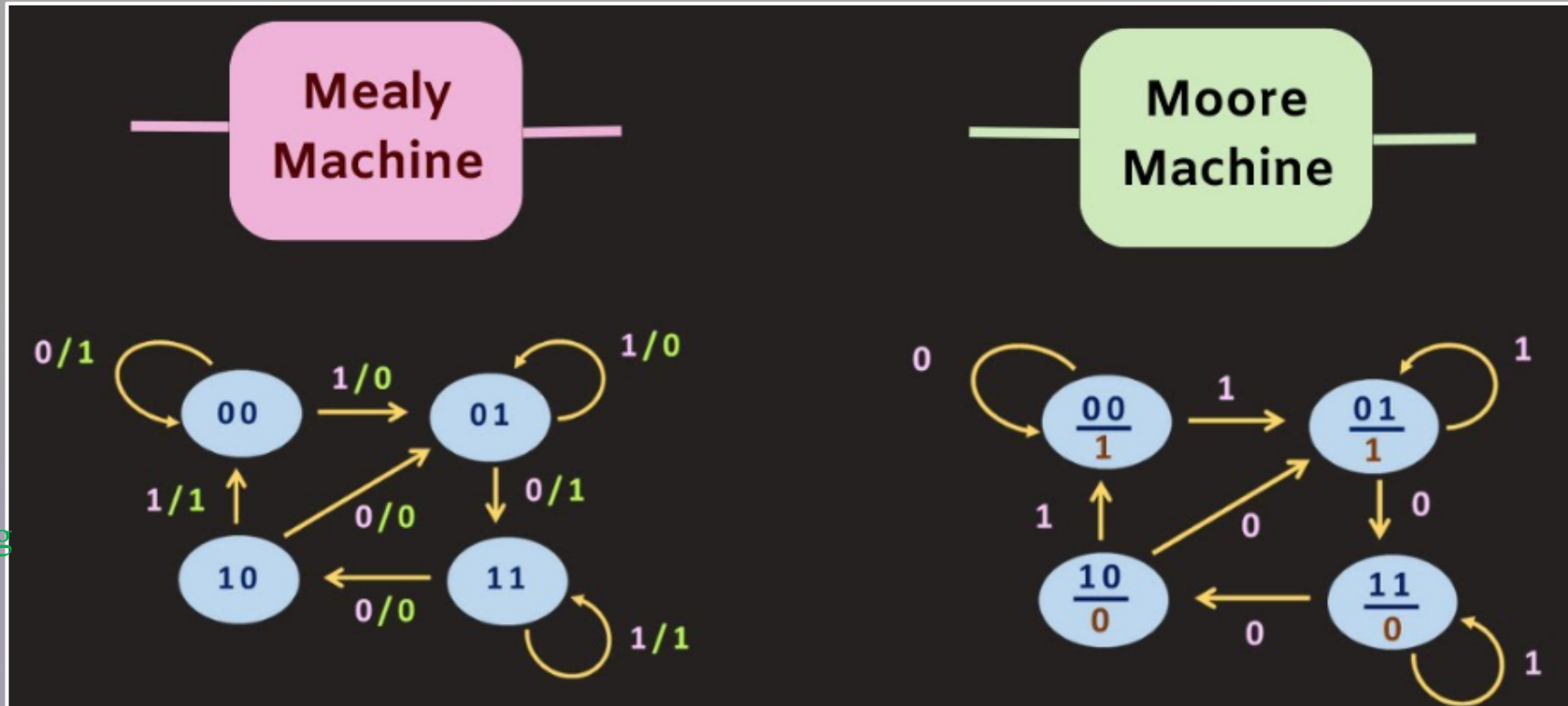


Lab Deliverables

1. State Transition Diagram

- A state transition diagram in an FSM visually represents the states, transitions, and conditions for moving between states.
- It shows the number of inputs, outputs, and the states of the FSM.
- It shows that how the change in input leads to the state transition, and the change in output.
- Every possible State is depicted as circles and represent different conditions or configurations of the system.
- Transitions are arrows between states, showing how the system moves from one state to another based on input conditions.
- Labels on the transitions specify the input that triggers the transition. In Mealy machines, outputs can also be labeled on transitions, while in Moore machines, outputs are associated with states.

1. State Transition Diagram



0/1:

0 – represents the input which leads to state transition.

1 – represents output of the circuit corresponding to the specific input.

1 – represents output of the circuit corresponding to the specific state.

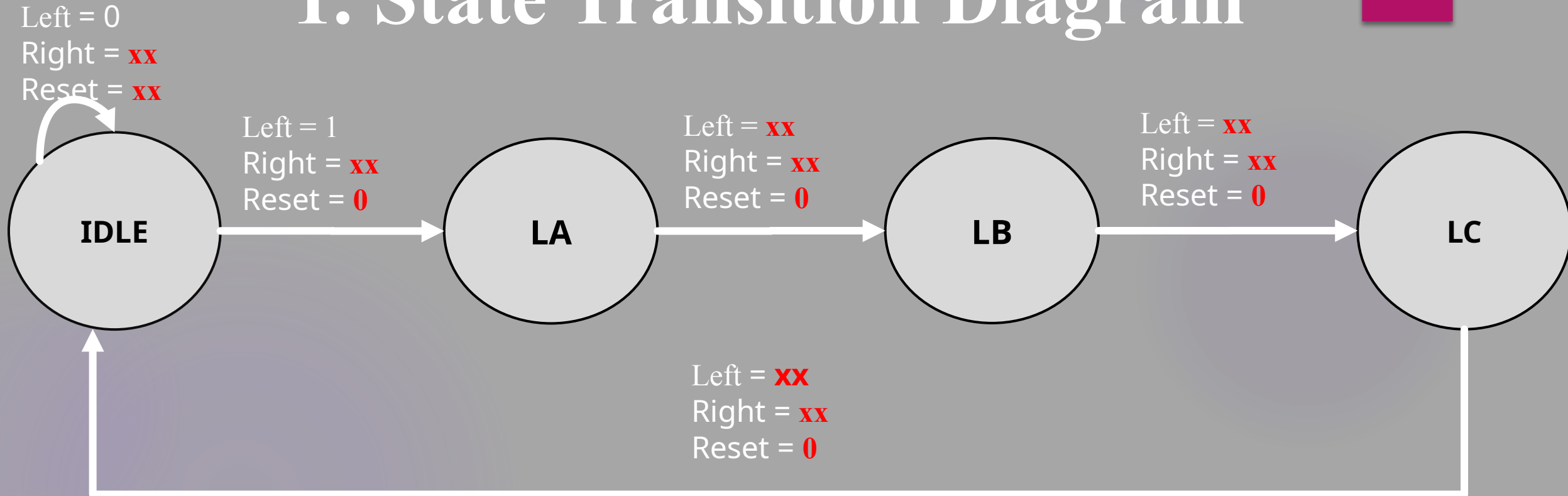
Mealy Machine

- Current state: 00
- Input: 1
- Then circuit's output will be 0 and it will go to state 01.

Moore Machine

- Current state: 00 with output 1
- Input: 1
- Then circuit will go to state 01 and output will be 1.

1. State Transition Diagram



Where
Left = **xx**
means Left = 0 or Left = 1 – don't care condition

- Make sure that you consider all possible inputs of the machine for each transition.
- Think about the output if input Reset becomes 1 in each transition.

2. State Assignment

- To differentiate between the states, each state is assigned a specific value.
- Typically, each state is assigned a binary value.
- If an FSM has N **total states**, the **minimum number of bits** B required to represent each state is given by:

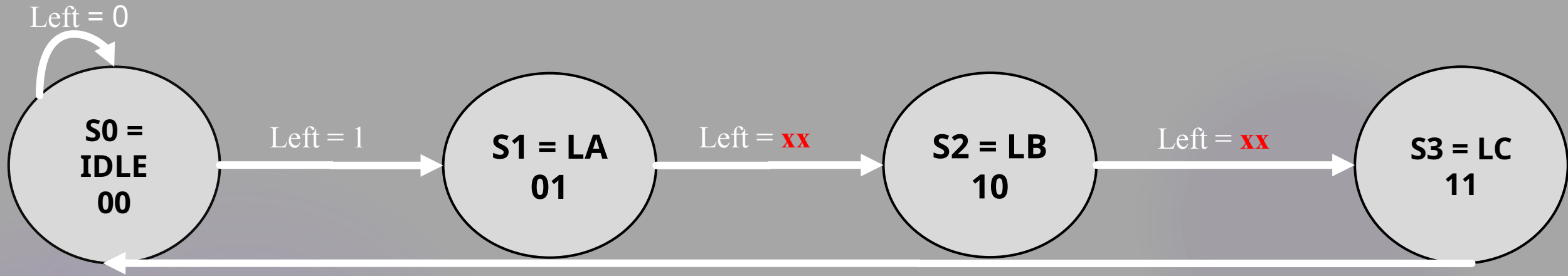
$$B = \log_2(N)$$

where:

N = total number of states

B = number of bits needed to uniquely encode each state

2. State Assignment



Left = **xx**

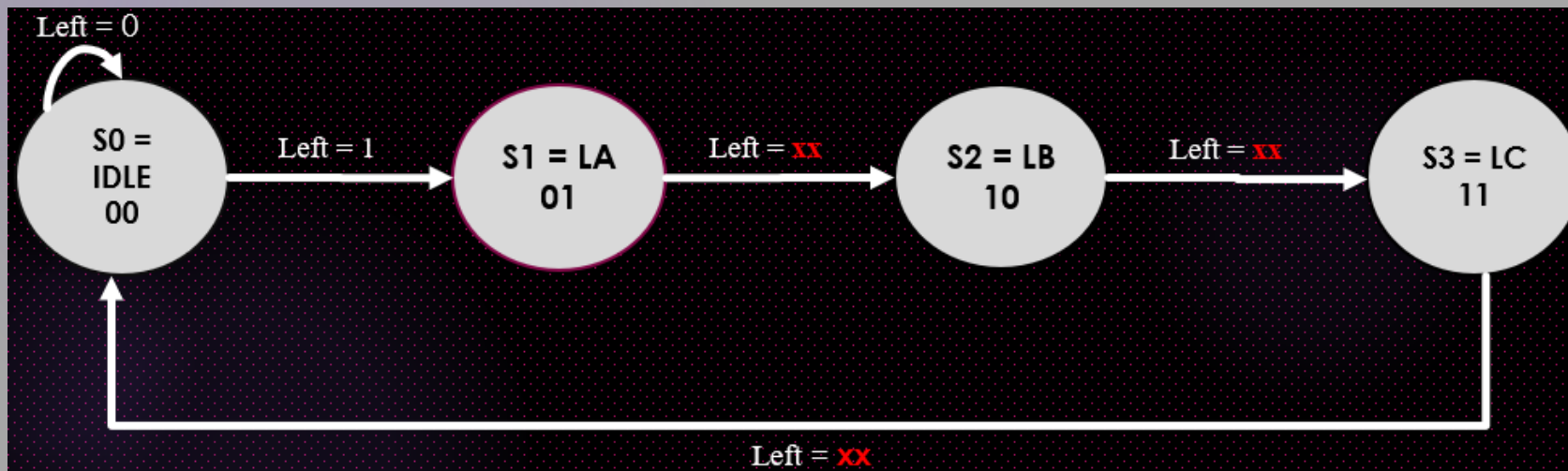
S0 → Idle State → All lights off → 00
S1 → LA State → LA on → 01
S2 → LB State → LA, LB on → 10
S3 → LC State → LA, LB, LC on → 11

3. State Transition Table

- A state transition table is a tabular representation of the behavior of a FSM.
- It helps map out how the FSM transitions between states in response to inputs.
- It lists:
 1. *Current State* - the state the FSM is in.
 2. *Input* - possible inputs or conditions.
 3. *Next State* - the state to transition to based on the current state and input.
 4. *Output* (for Mealy machines) - the output associated with the current state and input.

3. State Transition Table

Current State		Name of Current State	Input	Output			Next State		Name of Next State
A	B			LA	LB	LC	A	B	
0	0	S0 = IDLE	0	0	0	0	0	0	S0 = IDLE
0	0	S0 = IDLE	1	1	0	0	0	1	S1 = LA
0	1	S1 = LA	0	1	1	0	1	0	S2 = LB
0	1	S1 = LA	1	1	1	0	1	0	S2 = LB
1	0	S2 = LB	0	1	1	1	1	1	S3 = LC
1	0	S2 = LB	1	1	1	1	1	1	S3 = LC
1	1	S3 = LC	0	0	0	0	0	0	S0 = IDLE
1	1	S3 = LC	1	0	0	0	0	0	S0 = IDLE



Make sure that you consider all possible inputs of the machine for each transition in the table.

Lab Deliverables

- 4.1. I/O Equations using State Transition Table: Using K-map (you can use website shared before)
- 4.2. Circuit Diagram
5. Verilog Design Module Code – must include clk & reset as inputs
6. Clock implementation – The clock available on Basys-3 board is 100 MHz, means that every clock period is 10 nanoseconds long. The entire blinking sequence would be then 40 ns. This is a very short time and we need to slow down the circuit. For example, if you want to complete the entire flash sequence in 1 second, you need to generate a clock of 1 Hz. Refer to Lab 10 to design clock of desired frequency.
7. Verilog design module code of Top Level – Implement FSM along with clock divider.
8. Constraint File
9. Picture of FPGA board